

Android Security Assessment

OWASP UnCrackable App — Level 1

1. introduction

Target: UnCrackable-Level1.apk

Platform: Android

Analysis Type: Static Analysis, Smali Analysis, APK Modification

The assessment focused on:

- Identifying root detection mechanisms.
- Identifying debugger/debuggable detection.
- Recovering the hardcoded secret through static analysis.
- Modifying the application's Smali code to bypass root/debuggable checks.
- Rebuilding and signing the modified APK.
- Installing and validating the modified APK in an Android 9 environment.

2. overview

2.1 environment setup

Test Environment: Android 9

Analysis Environment: Kali Linux

The Android application was analyzed inside an isolated Android 9 environment running through Virtual Master. The Kali Linux host communicated with the Android environment through ADB over TCP/IP.

2.2 tools

Tool	Purpose
JADX	Java/decompiled code analysis
Apktool	APK decoding, Smali analysis, rebuilding
apksigner	APK signing
ADB	Device communication and APK installation

Python	Independent verification of cryptographic logic
Virtual Master	Android 9 test environment

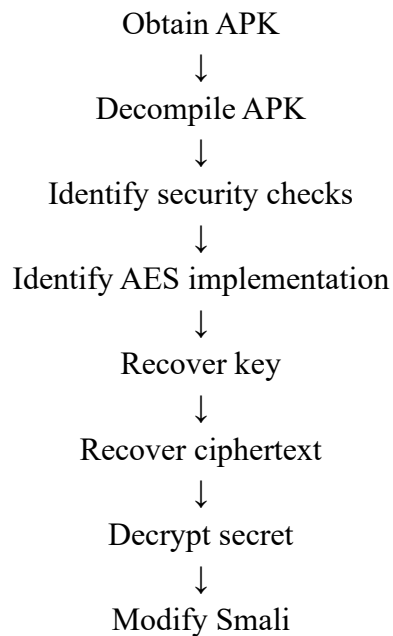
3. Findings

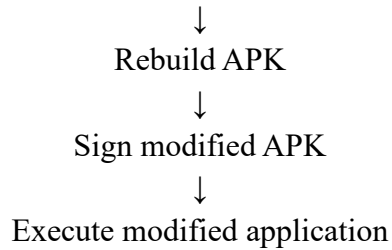
Finding	Severity	Result
Client-side root detection can be bypassed	Medium	Bypassed
Debuggable detection implemented entirely client-side	Low/Medium	Bypassed
Cryptographic key hardcoded in APK	High	Recovered
Encrypted secret recoverable through static analysis	High	Recovered
Secret validation performed locally	High	Successfully bypassed/reproduced

3.1 security impact

The primary security issue is that the application relies on client-side security controls and embedded cryptographic material.

An attacker with access to the APK can:





Therefore, security-sensitive secrets and trust decisions should not rely solely on client-side implementation.

4. Detail Findings

4.1 Initial Static Analysis

The original APK was analyzed using JADX.

The main application class identified was: `sg.vantagepoint.uncrackable1.MainActivity`

The `onCreate()` method contained two security mechanisms:

- Root detection
- Debuggable application detection

The relevant Java code was:

```
protected void onCreate(Bundle bundle) {  
    if (c.a() || c.b() || c.c()) {  
        a("Root detected!");  
    }  
    if (b.a(getApplicationContext())) {  
        a("App is debuggable!");  
    }  
    super.onCreate(bundle);  
    setContentView(R.layout.activity_main);  
}
```

This established that the application performs security checks before presenting the main interface.

4.2 Root Detection Analysis

The root detection implementation was located in: `sg.vantagepoint.a.c`

The class implemented three separate checks.

4.2.1 PATH-based su Detection

```

public static boolean a() {
    for (String str : System.getenv("PATH").split(":")) {
        if (new File(str, "su").exists()) {
            return true;
        }
    }
    return false;
}

```

This function iterates through directories contained in the system PATH and checks whether an executable named su exists.

4.2.2 Build Tags Detection

The second check examines Android build tags:

```

public static boolean b() {
    String str = Build.TAGS;
    return str != null && str.contains("test-keys");
}

```

The application considers the presence of: test-keys

as an indication that the device may be running a non-production or modified build.

4.2.3 Known Root File Detection

The third check searches for known root-related files:

```

public static boolean c() {
    for (String str : new String[] {
        "/system/app/Superuser.apk",
        "/system/xbin/daemonsu",
        "/system/etc/init.d/99SuperSUDaemon",
        "/system/bin/.ext/.su",
        "/system/etc/.has_su_daemon",
        "/system/etc/.installed_su_daemon",
        "/dev/com.koushikdutta.superuser.daemon/"
    }) {
        if (new File(str).exists()) {
            return true;
        }
    }
    return false;
}

```

The application checks several known paths associated with historical Android root-management tools.

4.3 Debuggable Detection

The second security mechanism was implemented in: `sg.vantagepoint.a.b`

The relevant code was:

```
public static boolean a(Context context) {  
    return (context.getApplicationContext()  
        .getApplicationInfo().flags & 2) != 0;  
}
```

The value: 2

corresponds to: `ApplicationInfo.FLAG_DEBUGGABLE`

Therefore, the application checks whether the APK was built with the Android debuggable flag enabled.

4.4 Secret Recovery

Further analysis revealed that the verification function performs AES decryption.

The relevant implementation was:

```
public static boolean a(String str) {  
    byte[] bArr;  
    try {  
        bArr = sg.vantagepoint.a.a.a(  
            b("8d127684cbc37c17616d806cf50473cc"),  
            Base64.decode(  
                "5UJiFctbmgbDoLXmpL12mkno8HT4Lv8dlat8FxR2GOc=",  
                0  
            )  
        );  
    } catch (Exception e) {  
        ...  
    }  
    return str.equals(new String(bArr));  
}
```

Two hardcoded values were identified:

- AES key: 8d127684cbc37c17616d806cf50473cc
- Base64 ciphertext: 5UJiFctbmgbDoLXmpL12mkno8HT4Lv8dlat8FxR2GOc=

The key is initially represented as hexadecimal and converted into bytes by the application's helper function.

4.5 AES Implementation Analysis

The encryption implementation was identified as:

```
SecretKeySpec secretKeySpec = new SecretKeySpec(bArr, "AES/ECB/PKCS7Padding");
Cipher cipher = Cipher.getInstance("AES");
cipher.init(2, secretKeySpec);
return cipher.doFinal(bArr2);
```

The important parameters were therefore:

Algorithm : AES

Mode : ECB

Operation : Decryption

Padding : PKCS7-compatible block padding

Because both the key and ciphertext are embedded in the application, the encryption does not prevent an analyst from recovering the secret.

The encryption does not need to be "cracked"; instead, the cryptographic implementation can be reproduced using the parameters recovered through reverse engineering.

4.7 Independent Secret Recovery with Python

The decryption process was reproduced independently using Python.

```
from Crypto.Cipher import AES
import base64

key = bytes.fromhex(
    "8d127684cbc37c17616d806cf50473cc"
)

ciphertext = base64.b64decode(
    "5UJiFctbmgbDoLXmpL12mkno8HT4Lv8dlat8FxR2GOc="
)

cipher = AES.new(key, AES.MODE_ECB)
plaintext = cipher.decrypt(ciphertext)
print(plaintext)
print(plaintext.decode())
```

The resulting plaintext was: I want to believe

This value was then independently tested against the application.

4.6 Smali Control-Flow Analysis

The Java-level analysis was then mapped to the Smali implementation.

The original onCreate() contained:

```
invoke-static {}, Lsg/vantagepoint/a/c;->a()Z
move-result v0
if-nez v0, :cond_0
invoke-static {}, Lsg/vantagepoint/a/c;->b()Z
move-result v0
if-nez v0, :cond_0
invoke-static {}, Lsg/vantagepoint/a/c;->c()Z
move-result v0
if-eqz v0, :cond_1
```

The important instructions were:

- if-nez
- if-eqz

These conditional branch instructions control whether execution reaches:

- :cond_0
- const-string v0, "Root detected!"
- invoke-direct {p0, v0}, ...

4.7 Rebuilding the Modified APK

After modifying the Smali code, the APK was rebuilt using Apktool 3.0.3:

```
apktool b apktool-output -o UnCrackable-Level1-unsigned.apk
```

The build completed successfully:

Built apk into: UnCrackable-Level1-unsigned.apk

The resulting APK was unsigned.

4.8 APK Signing

A dedicated Android lab keystore was created for signing the modified APK.

The APK was then signed using the official Android apksigner:

```
apksigner sign \  
--ks android-lab.keystore \  
--ks-key-alias android-lab \  
--out UnCrackable-Level1-signed.apk \  
UnCrackable-Level1-unsigned.apk
```

4.9 Dynamic Validation

The signed APK was installed into the Android 9 testing environment:

```
adb -s 192.168.100.12:10000 install -r \  
UnCrackable-Level1-signed.apk
```

Installation completed successfully.

The modified application was then tested.

4.10.1 Test 1 — Root Detection

Expected before patch: Root detected!

After patch: Root detection bypassed

4.10.2 Test 2 — Debuggable Detection

Expected before patch: App is debuggable!

After patch: Debuggable detection bypassed

4.10.3 Test 3 — Incorrect Secret

Input: Incorrect value

Result: Nope...

4.10.4 Test 4 — Recovered Secret

Input: I want to believe

Result: Success!

5. Recommendation

For a production Android application:

- Do not hardcode sensitive secrets, avoid embedding cryptographic keys directly inside the APK.
- Do not rely on client-side checks for authorization, root detection and similar mechanisms can be modified or bypassed.

- Move sensitive verification to a trusted backend, the client should not be considered a trusted execution environment.
- Use Android cryptographic facilities appropriately, when cryptographic keys genuinely need to be stored on-device, consider Android Keystore and appropriate key-management architecture. However, this should not be treated as an absolute protection against a sufficiently capable local attacker.
- Apply code hardening, obfuscation can increase the cost of reverse engineering, but it should be considered a defense-in-depth measure, not a replacement for secure architecture.

6. Conclusion

The UnCrackable-Level1 application was successfully analyzed using static analysis, Smali analysis, APK modification, and dynamic testing. The assessment demonstrated that:

- Root detection mechanisms could be identified through static analysis.
- Debuggable detection could be identified and bypassed.
- The application's Smali control flow could be modified to bypass the security checks.
- The AES key and ciphertext could be recovered from the APK.
- The encrypted secret could be independently recovered using Python.
- The modified APK could be rebuilt and cryptographically signed.
- The modified application could be installed and executed successfully on Android 9.
- The recovered secret successfully produced the application's Success response.
- An APK distributed to an attacker must be treated as an untrusted environment. Client-side logic, hardcoded secrets, and local security checks can be inspected and modified.